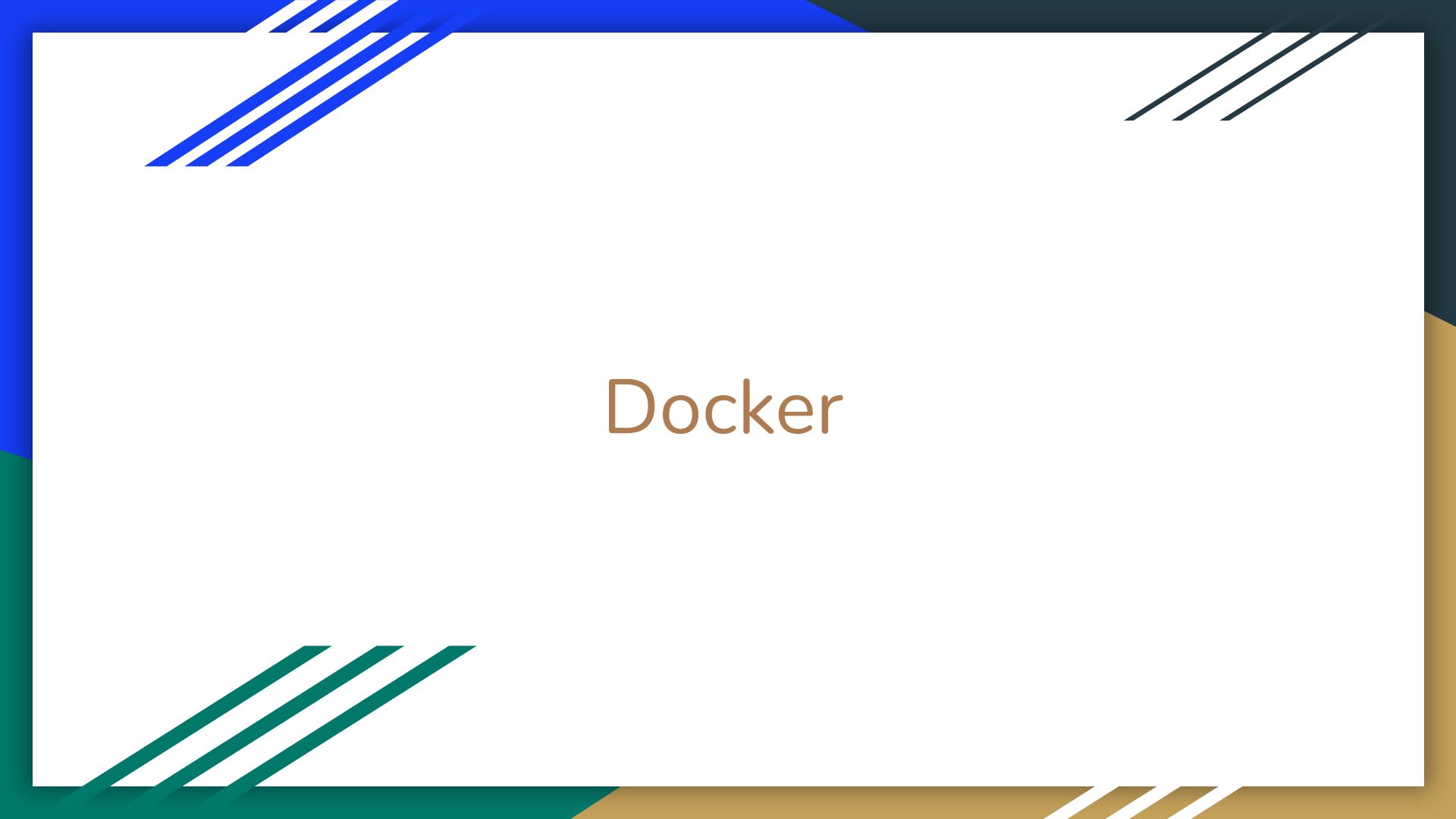




Docker

Qu'est-ce qu'un container

Un conteneur correspond à un empaquetement d'un environnement contenant d'une application ou un service, ses dépendances et sa configuration. Ce paquet s'appelle une image et permet d'être redéployé en quelques instants.

Pourquoi et comment utiliser un conteneur

La gestion des images va permettre de scripter facilement un environnement, de pouvoir le visionner sur git puisqu'il s'agit d'un fichier de code facilement maintenable, et de le déployer de manière très rapide.

Le but est d'isoler une application sur un environnement particulier pour pouvoir l'exécuter sur n'importe quel OS (Windows, linux, mac etc) sans qu'il y ait des problèmes d'adaptabilité, toujours avec exactement la même configuration.

Pour se faire, on utilise Docker, un outil permettant de créer les conteneurs, et de lire des images, appelées les Dockerfile.

Et avant les conteneurs ?

Les conteneurs sont assez récents. La notion existait sous Linux depuis 2008 mais étaient méconnus jusqu'à l'émergence des conteneurs Docker en 2013.

Ainsi, avant d'arriver à créer des images Docker, on utilisait des machines virtuelles (VM). Une VM va contenir un système d'exploitation dans son intégralité c'est-à-dire les pilotes, les fichiers binaires, les bibliothèques alors que les conteneurs Docker ne possèdent que le strict nécessaire pour l'exécution de l'application. Il fait appel à l'OS de sa machine hôte pour réaliser ses appels système et exécuter ses applications, d'où son extrême légèreté.

Pourquoi utiliser Docker ?

Docker répond à une problématique forte dans le monde du développement.

Prenons un exemple : vous avez développé votre projet de Twitter Lite en local. Tout fonctionne bien, mais au moment de mettre en production, vous vous rendez compte que vous ne savez pas comment déployer votre projet. Un autre exemple : vous êtes dans une équipe de 10 personnes et chacun utilise un OS différent (Ubuntu, macOS, Windows, CentOS, etc.). Comment faire pour avoir un environnement unifié et fonctionnel chez l'ensemble des développeurs ?

Docker répond à ces problématiques en créant des conteneurs. Grâce à Docker, vous n'aurez plus de problème de différence d'environnement, et votre code marchera partout !

Utilisez Docker sur tous vos environnements

Docker est utilisé par de très nombreuses sociétés pour différents usages. Ainsi, un des premiers usages de Docker se trouve dans la création d'environnements locaux. Il est plus simple d'utiliser Docker en local quand on travaille avec de nombreuses versions différentes des logiciels, et ainsi ne pas avoir de problèmes de compatibilité entre elles.

On retrouve aussi Docker dans les domaines de la CI (Continuous Integration, ou Intégration Continue) et de la CD (Continuous Delivery, ou Livraison Continue). Cela permet à la CI/CD de créer rapidement des espaces isolés pour faire tourner vos tests.

Les conteneurs Docker

Un conteneur Docker est un environnement d'exécution contenant tous les composants nécessaires, tels que le code, les dépendances et les bibliothèques, nécessaires pour exécuter le code de l'application sans utiliser les dépendances de la machine hôte. Ce moteur d'exécution de conteneur s'exécute sur le moteur d'un serveur, d'une machine ou d'une instance cloud. Le moteur exécute plusieurs conteneurs en fonction des ressources sous-jacentes disponibles.

Les images

Une image Docker, ou image de conteneur, est un fichier exécutable autonome utilisé pour créer un conteneur. Cette image de conteneur contient toutes les bibliothèques, dépendances et fichiers dont le conteneur a besoin pour s'exécuter. Une image Docker est partageable et portable. Vous pouvez donc déployer la même image à plusieurs emplacements à la fois, comme dans un fichier binaire logiciel.

Registre d'images

Vous pouvez stocker des images dans des registres pour suivre les architectures logicielles complexes, les projets, les secteurs d'activité et les accès des groupes d'utilisateurs. Par exemple, le registre public du Docker Hub contient des images telles que des systèmes d'exploitation, des cadres de langage de programmation, des bases de données et des éditeurs de code.

Orchestrer

Pour déployer et scaler un ensemble de conteneurs afin de communiquer efficacement entre différentes machines ou machines virtuelles, vous avez besoin d'une plateforme d'orchestration de conteneurs telle que Kubernetes, Docker Compose ou Docker Swarm

Comment installer docker

Linux :

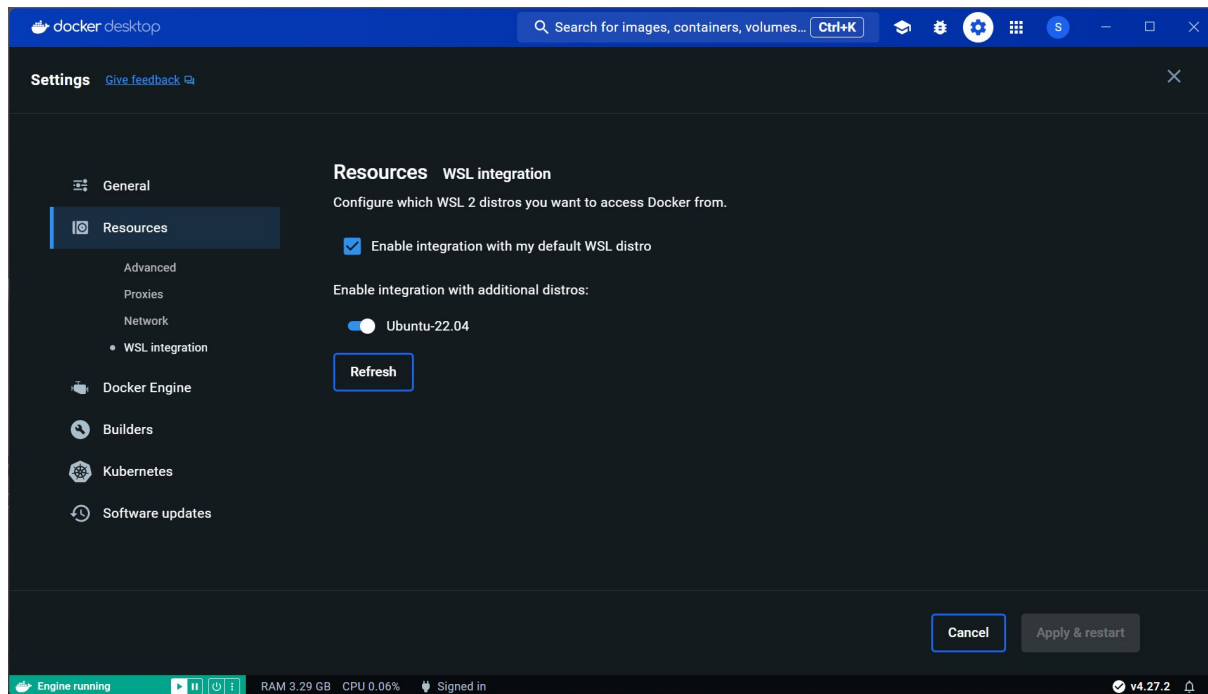
```
sudo apt install docker.io
```

```
sudo apt install docker-compose
```

Windows & IOS : télécharger l'installateur de Windows ou IOS et suivre les instructions

Environnement windows

Pour utiliser docker, il faut activer le WSL Integration, en téléchargeant et installant auparavant un environnement ubuntu depuis le [MS Store](#)



Librairie d'image

Docker met à disposition une très large bibliothèque d'image officielle à travers leur site

<https://hub.docker.com/>

Bien sûr, la communauté des développeurs n'est pas en reste et a créé une multitude d'images disponibles sur le net

Quelques commandes docker utiles

- `docker run <nom_image>` : Démarrer un conteneur
- `docker ps` : Récupérer la liste des conteneurs en cours d'exécution
- `docker ps -a` : Récupérer la liste de tous les conteneurs (exécutés ou non)
- `docker stop <id_conteneur>` : Arrêter un conteneur à partir de son id
- `docker restart <id_conteneur>` : Redémarrer un conteneur à partir de son id
- `docker rm <id_conteneur>` : Supprimer un conteneur à partir de son id
- `docker pull <nom_image>` : Télécharger une image depuis la librairie Docker
- `docker pull <nom_image>:<numero_de_version>` : Télécharger une version spécifique d'une image
- `docker images` : Récupérer la liste des images téléchargées
- `docker rmi <id_image>` : Supprimer une image

Docker Hub

Docker Hub est une plateforme en ligne dédiée au stockage, au partage et à la distribution d'images de conteneurs. Il s'agit du plus grand dépôt public d'images de conteneurs, permettant aux utilisateurs de trouver et d'utiliser des images créées par la communauté ou des éditeurs officiels. En plus de proposer des images publiques, Docker Hub offre également la possibilité de créer des dépôts privés, réservés à un usage interne.

Historique de Docker Hub

Lancé en 2013, Docker Hub a vu le jour peu après la création de Docker, afin de répondre à un besoin grandissant : la distribution et la gestion d'images de conteneurs. À cette époque, Docker commençait à révolutionner la façon dont les développeurs déployaient leurs applications en utilisant des conteneurs légers. Cependant, il fallait une solution simple pour partager ces images. C'est là qu'intervient Docker Hub, offrant un dépôt centralisé pour héberger et distribuer ces conteneurs.

Au début, Docker Hub servait principalement de bibliothèque publique pour des images de base, telles que Ubuntu, Nginx, ou encore des bases de données populaires comme Postgresql. Les développeurs pouvaient ainsi gagner du temps en téléchargeant des images préconfigurées, plutôt que de devoir configurer chaque composant à la main.

Historique de Docker Hub

Au fil des années, avec la montée en popularité de Docker et des pratiques DevOps (DevOps), Docker Hub a évolué. Il ne se contente plus d'être un simple registre public. Il permet désormais aux entreprises de créer des dépôts privés pour protéger leurs images internes et propose des fonctionnalités avancées. Cela a rendu Docker Hub non seulement utile pour les projets personnels, mais aussi pour les déploiements à grande échelle.

Avec la montée en puissance des orchestrateurs de conteneurs tels que Kubernetes et des outils comme Helm, Docker Hub est devenu une plateforme incontournable pour les équipes cherchant à automatiser et simplifier le cycle de vie de leurs applications conteneurisées.

Créer une base de donnée

On va alors pouvoir utiliser une image existante dans la librairie de Docker, dont on peut trouver les différentes versions.

Partons sur une base de donnée mysql ici : https://hub.docker.com/_/mysql

Les versions

Comme sur un registry npm, plusieurs versions sont disponibles par image

Supported tags and respective Dockerfile links

- `8.3.0`, `8.3`, `8`, `innovation`, `latest`, `8.3.0-oraclelinux8`, `8.3-oraclelinux8`, `8-oraclelinux8`, `innovation-oraclelinux8`, `oraclelinux8`, `8.3.0-oracle`, `8.3-oracle`, `8-oracle`, `innovation-oracle`, `oracle`
- `8.0.36`, `8.0`, `8.0.36-oraclelinux8`, `8.0-oraclelinux8`, `8.0.36-oracle`, `8.0-oracle`
- `8.0.36-bullseye`, `8.0-bullseye`, `8.0.36-debian`, `8.0-debian`

“Cloner” une image

Pour pouvoir utiliser une image de la librairie docker, on va d’abord la récupérer avec la commande pull

```
docker pull mysql:9.3
```

Lancer une image

```
docker run --name ynov-ci-mysql -e MYSQL_ROOT_PASSWORD=my-secret-pw -d mysql
```

“ynov-ci-mysql” est le nom que vous souhaitez attribuer à votre conteneur, “my-secret-pw” est le mot de passe à définir pour l'utilisateur root MySQL

Cette commande va vous afficher un container id

Entrer dans le container

`docker exec -it ynov-ci-mysql bash`

`bash-4.4# mysql -u root -p`

Entrez votre mot de passe

A cette étape vous pouvez alors créer votre base de donnée, vos tables etc

Structure d'un Dockerfile

Un Dockerfile est un fichier texte qui décrit, étape par étape, comment construire une image de conteneur. Chaque ligne du Dockerfile correspond à une instruction qui précise une action à exécuter pour configurer l'image. Ces instructions suivent une syntaxe simple : Instruction suivie d'un ou plusieurs arguments, qui définissent les détails de l'action.

Commentaire

INSTRUCTION argument1 argument2

L'instruction FROM

Pour qu'un fichier Dockerfile soit valide, il doit commencer avec l'instruction FROM qui définit l'image de base. Cette image de base peut être n'importe quelle image valide.

L'argument facultatif --platform permet de spécifier la plate-forme de l'image dans le cas où FROM fait référence à une image multiplateforme. Par exemple, linux/amd64. Par défaut, la plateforme utilisée est celle où est effectuée la construction de l'image.

Les valeurs tag ou digest sont facultatives. Si vous omettez l'un ou l'autre, le générateur suppose que vous voulez récupérer l'image possédant le tag latest.

```
FROM [--platform=<platform>] <image> [AS <name>]
```

ou

```
FROM [--platform=<platform>] <image>[:<tag>] [AS <name>]
```

ou

```
FROM [--platform=<platform>] <image>[@<digest>] [AS <name>]
```


L'instruction ARG

L'instruction ARG est la seule instruction qui peut précéder l'instruction FROM. Elle permet de définir des variables qui peuvent être transmises au moment de la construction de l'image : ARG <name>[=<default value>]

Il est possible de définir des valeurs par défaut avec l'opérateur = suivi de cette valeur.

```
ARG version=1.15.3-alpine@sha256:829a63ad2b1389e393e5decf5df25860347d09643c335d1dc3d91d25326d3067
```

Pour utiliser la variable, on utilise la même syntaxe qu'on utilise dans les scripts shell : RUN echo \${VERSION}

Les instructions FROM peuvent utiliser des variables déclarées avant elle.

```
ARG version=1.15.3-alpine@sha256:829a63ad2b1389e393e5decf5df25860347d09643c335d1dc3d91d25326d3067
```

```
FROM nginx:${VERSION}
```

L'instruction ENV

Contrairement à ARG, l'instruction ENV permet de définir des variables d'environnement qui seront utilisés dans les commandes exécutées avec l'instruction RUN que nous verrons juste après. Ces variables sont persistantes et seront disponibles au moment de l'exécution de l'image de conteneur.

ENV <key>=<value> ...

Par exemple pour une image proposant un service Postgres la variable d'environnement PGDATA qui indique où se trouvent les données des bases.

ENV PGDATA=/data

Elles peuvent utiliser des variables définies avec ARG.

ARG PG_VERSION=9.3.4

ENV PG_VERSION=\${PG_VERSION}

L'instruction RUN

L'instruction RUN permet de lancer des commandes aux moments de la construction de l'image.

Par exemple pour installer des packages depuis une image de base Alpine :

RUN apk update

RUN apk add nginx

Optimisation de RUN

Pour optimiser la taille des images, car chaque instruction RUN ajoute une couche à l'image résultante, il est conseillé d'enchaîner les commandes en les séparant avec la séquence `&&`.

```
RUN apk update && apk add nginx
```

Lorsque beaucoup de commande s'enchainent, on peut utiliser un antislash `\` pour poursuivre sur la ligne suivante.

```
RUN apk update &&\
```

```
    apk add nginx
```

Lors de la construction de l'image, pour simplifier le debug, je vous conseille d'utiliser plusieurs instructions RUN. Une fois l'image au point, regrouper les !

L'instruction COPY

L'instruction COPY permet d'intégrer des fichiers et des dossiers pour les ajouter au système de fichiers de l'image dans le répertoire <dest>.

```
COPY [--chown=<user>:<group>] [--chmod=<perms>] <src>... <dest>
```

```
COPY [--chown=<user>:<group>] [--chmod=<perms>] ["<src>",... "<dest>"]
```

La seconde forme est à utiliser si vous avez des espaces dans les noms de fichiers. Comme vous pouvez le remarquer, on peut indiquer plusieurs sources dans une seule instruction.

```
COPY . /app
```

Attention les arguments optionnels --chown et --chmod ne fonctionnent que sur des images de conteneurs Linux.

L'instruction ADD

L'instruction ADD fonctionne comme l'instruction COPY sauf qu'il permet d'intégrer des URL de fichiers distants et/ou des archives qui seront décompressés à la volée et les ajoute au système de fichiers de l'image dans le répertoire <dest>.

```
ADD [--chown=<user>:<group>] [--chmod=<perms>] [--checksum=<checksum>] <src>... <dest>
```

```
ADD [--chown=<user>:<group>] [--chmod=<perms>] ["<src>",... "<dest>"]
```

Vous l'aurez remarqué les instructions ADD et COPY sont fonctionnellement similaires. Mais il est conseillé de privilégier COPY parce qu'elle est plus transparente que ADD. Par contre, si vous devez ajouter une archive ou une URL distante alors faites appel à l'instruction ADD

L'instruction USER

L'instruction USER définit l'utilisateur ou l'UID et éventuellement le groupe d'utilisateurs ou le GID à utiliser pour le reste de l'étape en cours. L'utilisateur spécifié est utilisé pour les instructions RUN et, au moment de l'exécution de l'image de conteneur.

```
USER <user>[:<group>]
```

#ou

```
USER UID[:GID]
```

Exemple User

FROM alpine@sha256:d7342993700f8cd7aba8496c2d0e57be0666e80b4c441925fc6f9361fa81d10e

ARG USER_UID=1000

ARG USER_GID=\${USER_UID}

RUN groupadd --gid \$USER_GID \$USERNAME \

&& useradd --uid \$USER_UID --gid \$USER_GID -m \$USERNAME \

&& apt update && apt install -y sudo \

&& echo \$USERNAME ALL=\\(root\\) NOPASSWD:ALL > /etc/sudoers.d/\$USERNAME \

&& chmod 0440 /etc/sudoers.d/\$USERNAME

USER \$USERNAME

L'instruction WORKDIR

L'instruction WORKDIR définit le répertoire de travail pour toutes les instructions qui la suivent dans le Dockerfile. Si le répertoire n'existe pas, il sera créé.

```
WORKDIR /path/to/workdir
```

L'instruction EXPOSE

L'instruction EXPOSE informe le moteur de conteneur que le conteneur écoute sur les ports réseau spécifiés au moment de l'exécution. Vous pouvez spécifier le protocole TCP ou UDP, TCP étant la valeur par défaut.

EXPOSE <port> [<port>/<protocol>...]

L'instruction EXPOSE ne publie pas réellement le port. Il est utilisé à des fins de documentations. Il faudra le spécifier dans les arguments de la commande lançant l'image de conteneur.

Pour réellement publier un port (c'est-à-dire le rendre accessible depuis l'extérieur du conteneur), vous devez le spécifier lorsque vous lancez le conteneur, généralement avec l'option -p ou --publish de la commande docker run → docker run -p 8080:80 votre_image

Les instructions CMD et ENTRYPOINT

Ces deux instructions permettent de définir les commandes qui seront exécutées au moment du lancement de l'image de conteneur.

La principale différence entre CMD et ENTRYPOINT est que les commandes fournies par CMD peuvent être remplacés, alors que celles fournies par ENTRYPOINT ne le peuvent pas à moins de les forcer avec le bon argument dans la CLI de votre moteur de conteneur.

```
CMD ["executable","param1","param2"]
```

```
ENTRYPOINT ["executable", "param1", "param2"]
```

Exemple CMD et ENTRYPOINT

FROM debian:stable

RUN apt update && apt install -y --force-yes apache2

EXPOSE 80 443

VOLUME ["/var/www", "/var/log/apache2", "/etc/apache2"]

ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]

Appliquer un script SQL

Mettons que nous souhaitons créer un environnement mysql avec une base de donnée crée par défaut

On va alors créer notre propre image, qui utilisera mysql et qui lancera un script sql

La base de donnée

Créons un fichier migration-v001.sql qui contiendra notre nouvelle base de donnée:

```
CREATE DATABASE ynov_ci;
```

Dockerfile

On va créer notre première image. Par convention, on crée un fichier Dockerfile, qui sera chargé par défaut.
On pourrait lui donner un autre nom et utiliser une option pour préciser le chemin

Dockerfile

```
#Create MySQL Image for JSP Tutorial Application
```

```
FROM mysql:9.2
```

```
ENV MYSQL_ROOT_PASSWORD=ynovpwd
```

```
COPY ./sqlfiles/migration-v001.sql /docker-entrypoint-initdb.d
```

```
EXPOSE 3306
```


Build notre image

On va utiliser notre dockerfile pour créer une image docker

`docker build -t migration_mysql .`

Ici on demande à docker de build une image, en la nommant `migration_mysql`, et en l'appliquant sur le dossier courant. Il ira donc chercher le fichier Dockerfile dans le dossier courant.

Voir nos images

```
[loisefenoll@MacBook-Air-de-Loise ci-cd-ynov % docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
migration_mysql     latest             87c509558958       24 seconds ago     638MB
```

On peut également voir la liste des images à travers docker desktop

 Containers

 Images

 Volumes

 Builds NEW

 Dev Environments BETA

 Docker Scout

Extensions ⋮

 Add Extensions

Images [Give feedback](#)

Local Hub

0 Bytes / 638.38 MB in use 1 images

Last refresh: 18 hours ago 

 Search



☐	Name	Tag	Status	Created	Size	Actions
☐	migration_mysql 87c509558958 	latest	Unused	1 second ago	638.38 MB	  

Showing 1 item

Walkthroughs

```
1 FROM node
2 RUN mkdir -p
3 WORKDIR /app
4 COPY packa
```

How do I run a container?

6 mins



Run Docker Hub images

5 mins

Lancer un container

```
docker run -d --name migration_container migration_mysql
```

On donne à notre container le nom `migration_container` et on lui demande de créer le container à partir de l'image `migration_mysql` et de s'exécuter en détaché (en arrière plan)

Containers

Images

Volumes

 Builds NEW Dev Environments BETA

Docker Scout

Extensions ⋮

Add Extensions

Containers [Give feedback](#)

Container CPU usage ⓘ

25.46% / 800% (8 CPUs available)

Container memory usage ⓘ

368.8MB / 3.74GB

[Show charts](#)

Search



Only show running containers



Name

Image

Status

CPU (%)

Port(s)

Last started

Actions

[migratio](#)

37135ba5

[migration_mys](#)

Running

25.46%

53 seconds ago



Showing 1 item

Walkthroughs ×

Multi-container applications

8 mins

\$ docker init

Containerize your application

3 mins

[View more in the Learning center](#)

Containers

Images

Volumes

Builds **NEW**Dev Environments **BETA**

Docker Scout

Extensions

+ Add Extensions

migration_container

[migration_mysql](#)

37135ba521d4

STATUS

Running (1 minute ago)



Logs

Inspect

Bind mounts

Exec

Files

Stats

```
2024-02-20 11:28:17 2024-02-20T10:28:17.740374Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version:
'8.3.0' socket: '/var/run/mysqld/mysqld.sock' port: 0 MySQL Community Server - GPL.
2024-02-20 11:28:18 Warning: Unable to load '/usr/share/zoneinfo/iso3166.tab' as time zone. Skipping it.
2024-02-20 11:28:18 Warning: Unable to load '/usr/share/zoneinfo/leap-seconds.list' as time zone. Skipping it.
2024-02-20 11:28:18 Warning: Unable to load '/usr/share/zoneinfo/leapseconds' as time zone. Skipping it.
2024-02-20 11:28:18 Warning: Unable to load '/usr/share/zoneinfo/tzdata.zi' as time zone. Skipping it.
2024-02-20 11:28:18 Warning: Unable to load '/usr/share/zoneinfo/zone.tab' as time zone. Skipping it.
2024-02-20 11:28:18 Warning: Unable to load '/usr/share/zoneinfo/zone1970.tab' as time zone. Skipping it.
2024-02-20 11:28:18 2024-02-20T10:28:18.833303Z 11 [System] [MY-013172] [Server] Received SHUTDOWN from user root. Shutting down
mysqld (Version: 8.3.0).
2024-02-20 11:28:20 2024-02-20T10:28:20.599903Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 8.3.0
) MySQL Community Server - GPL.
2024-02-20 11:28:20 2024-02-20T10:28:20.599937Z 0 [System] [MY-015016] [Server] MySQL Server - end.
2024-02-20 11:28:21 2024-02-20T10:28:20.880459Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2024-02-20 11:28:21 2024-02-20T10:28:21.125228Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.3.0) starting as proce
ss 1
2024-02-20 11:28:21 2024-02-20T10:28:21.131830Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2024-02-20 11:28:21 2024-02-20T10:28:21.275171Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2024-02-20 11:28:21 2024-02-20T10:28:21.708984Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2024-02-20 11:28:21 2024-02-20T10:28:21.709049Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Enc
rypted connections are now supported for this channel.
2024-02-20 11:28:21 2024-02-20T10:28:21.716846Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location
'/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
2024-02-20 11:28:21 2024-02-20T10:28:21.743369Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::
' port: 33060, socket: /var/run/mysqld/mysqlx.sock
2024-02-20 11:28:21 2024-02-20T10:28:21.743413Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version:
'8.3.0' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.
```

Entrer dans le container

```
docker exec -it migration_container /bin/bash
```

```
mysql -p
```

```
//on entre le mode de passe ynovpwd
```

```
show databases;
```

```
//on doit voir la base de donné ynov_ci
```

Sécurisons un peu

On va maintenant supprimer la variable d'environnement en dur et la passer en paramètre au moment du run de l'image

Mais avant tout, on va couper notre container, le supprimer, et supprimer l'image qui contient cette erreur

```
docker container stop migration_container
```

```
docker container rm migration_container
```

```
docker image rm migration_mysql
```


Passer les variables d'environnement

Supprimons la ligne ENV MYSQL_ROOT_PASSWORD ynovpwd

On va rebuild notre image → `docker build -t migration_mysql .`

Puis on moment de créer et exécuter notre container, on va passer notre mot de passe dans un .env avec `--env-file`

`docker run -d --env-file .env --name migration_container migration_mysql`

Puis on reteste que tout fonctionne → `docker exec -it migration_container /bin/bash`

Multiple import de fichier sql

Notre fichier Dockerfile va importer mysql et va copier le script sql vers le dossier `/docker-entrypoint-initdb.d`, qui est créé par défaut dans tous les containers docker

Pour importer plusieurs fichiers sql, on va copier tout le dossier `sqlfiles` directement qui va contenir nos scripts.

Attention, les scripts sql vont s'appliquer dans l'ordre alphabetique, donc il faudra les nommer en conséquence.

Dans un deuxième script sql, on créera une table utilisateur.

A vous de jouer

Bonnes pratiques des Dockerfile

Pour écrire des Dockerfiles performants, maintenables et sécurisés, il est essentiel de suivre certaines bonnes pratiques. Ces pratiques vous aideront à optimiser vos images, réduire leur taille et minimiser les erreurs. En complément de ces bonnes pratiques, l'utilisation d'outils de contrôle vous permettra de perfectionner votre maîtrise de la création d'images Docker. Voici quelques conseils et outils incontournables pour progresser.

Écrire des Dockerfiles simples et lisibles

La simplicité est clé pour écrire des Dockerfiles maintenables. Utilisez des instructions claires et évitez les combinaisons complexes de commandes sur une seule ligne. Chaque commande doit être explicite et facile à comprendre pour quiconque lit le Dockerfile.

```
RUN apt-get update && apt-get install -y curl vim \
```

```
&& apt-get clean \
```

```
&& rm -rf /var/lib/apt/lists/*
```

Cette commande est claire, installe les paquets nécessaires et nettoie ensuite pour réduire la taille de l'image.

Minimiser la taille de l'image

Utiliser des images de base légères, comme alpine et combiner les commandes pour réduire le nombre de couches sont des stratégies essentielles pour minimiser la taille de l'image. Moins l'image est volumineuse, plus elle est rapide à déployer et à transférer.

```
FROM python:3.11-alpine
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

En utilisant alpine, une version allégée de Linux, vous réduisez considérablement la taille de l'image finale.

Utiliser .dockerignore

Comme pour .gitignore, un fichier .dockerignore permet d'exclure les fichiers inutiles lors de la construction de l'image, ce qui contribue à réduire sa taille et à éviter l'inclusion de données sensibles.

*.log

node_modules

.env

Ces fichiers ne seront pas inclus dans l'image Docker, ce qui la rendra plus légère et plus sécurisée.

Séparer les phases de construction et d'exécution

Utiliser une approche multi-étape pour séparer la phase de construction (qui inclut les outils et dépendances de développement) de la phase d'exécution permet de réduire la taille de l'image finale tout en améliorant la sécurité. Ici la compilation est effectuée dans une 1ere étape, puis le binaire compilé est copié dans une image alpine légère.

```
FROM golang:1.18 AS builder
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN go build -o myapp
```

```
FROM alpine:3.14
```

```
COPY --from=builder /app/myapp /usr/local/bin/myapp
```

```
ENTRYPOINT ["myapp"]
```

Gérer les variables d'environnement avec précaution

Définissez les variables d'environnement (ENV) et les arguments de construction (ARG) pour gérer les configurations, mais évitez d'y inclure des informations sensibles comme des clés API ou des mots de passe.

```
ARG APP_ENV=production
```

```
ENV APP_ENV=${APP_ENV}
```

Cette configuration permet de gérer facilement les environnements de développement et de production.

Utiliser des outils de contrôle

Pour garantir la qualité et l'efficacité de vos Dockerfiles, il est important d'utiliser des outils de contrôle qui vous aideront à respecter les bonnes pratiques et à optimiser vos images.

- Hadolint est un outil de linting qui analyse vos Dockerfiles et vous donne des recommandations pour améliorer leur qualité. Il vérifie si vous suivez les bonnes pratiques et vous alerte sur les erreurs courantes.
- dive est un outil d'analyse qui vous permet de plonger dans les couches de votre image Docker pour comprendre comment elle est construite. Il vous aide à identifier les couches inutiles et à optimiser la taille de votre image.
- Conteneur-Structure-Test est un outil qui permet de valider la structure de vos images Docker. Il peut vérifier l'exécution des commandes, les métadonnées, et le contenu du système de fichiers pour s'assurer que votre image est conforme à vos attentes.

Les fonctionnalités de Docker Hub

Docker Hub propose une large gamme de fonctionnalités qui facilitent la gestion des images Docker et améliorent le workflow de développement et de déploiement. Voici les principales caractéristiques qui rendent Docker Hub si populaire parmi les développeurs et les équipes DevOps.

Dépôts publics et privés

Docker Hub vous permet de créer des dépôts publics ou privés selon vos besoins. Les dépôts publics sont accessibles à tous, ce qui est pratique si vous voulez partager des images Docker avec la communauté ou d'autres équipes. Les dépôts privés, en revanche, sont protégés et accessibles uniquement aux personnes ou équipes que vous autorisez, offrant ainsi une solution sécurisée pour les entreprises ou projets sensibles.

Automated builds

L'une des fonctionnalités les plus puissantes de Docker Hub est la capacité à configurer des Automated Builds. Cette fonctionnalité permet d'intégrer Docker Hub avec un dépôt Git, comme GitLab ou GitHub, pour construire automatiquement une nouvelle image chaque fois qu'un changement est poussé dans le dépôt. Cela facilite l'intégration continue (CI), car à chaque mise à jour de code, une nouvelle version de l'image est automatiquement créée et publiée. Par exemple, en connectant Gitlab CI à Docker Hub, vous pouvez déclencher la construction d'une nouvelle image à chaque modification de votre projet.

Webhooks

Les Webhooks permettent d'automatiser certaines actions en envoyant des notifications lorsqu'un événement spécifique se produit, comme la publication d'une nouvelle version d'une image. Vous pouvez configurer Docker Hub pour notifier un système externe ou déclencher une action spécifique à chaque fois qu'une image est mise à jour, améliorant ainsi l'automatisation du cycle de vie de vos applications.

Équipes et organisations

Pour les projets collaboratifs, Docker Hub permet de créer des équipes et des organisations, facilitant ainsi la gestion des accès aux dépôts. Vous pouvez définir des rôles et des permissions spécifiques pour chaque membre de l'équipe, afin de garantir que seuls ceux qui en ont besoin peuvent modifier ou publier des images. Cela est particulièrement utile pour les entreprises qui gèrent plusieurs projets et veulent centraliser la gestion des accès aux images.

Comparaison entre les comptes gratuits et payants sur Docker Hub

Fonctionnalité	Compte Gratuit	Compte Payant (Pro, Team, Business)
Dépôts privés	1 dépôt privé maximum	Dépôts privés illimités
Limitation des pulls	100 pulls toutes les 6 heures par adresse IP	Limitation plus élevée ou pulls illimités
Automated Builds	Accès limité (fréquence et parallélisation réduites)	Parallélisation accrue, fréquence plus élevée
Stockage des images	Stockage limité (suffisant pour de petits projets)	Stockage illimité
Collaboration et gestion d'équipe	Partage basique des dépôts privés	Gestion avancée des équipes et organisations (rôles, permissions)
Support technique	Documentation et support communautaire	Support dédié et prioritaire (selon le niveau d'abonnement)

Création d'un dépôt sur Docker Hub

1. Connectez-vous à votre compte Docker Hub.
2. Cliquez sur Create Repository dans le tableau de bord.
3. Nommez le dépôt de manière unique (ex. : username/myapp).
4. Choisissez la visibilité : Public ou Privé.
5. Ajoutez une description pour expliquer le contenu de l'image.
6. Cliquez sur Create pour finaliser la création du dépôt.

Votre dépôt est maintenant prêt à accueillir vos images.

Pusher une image vers Docker Hub

1. Se connecter à Docker Hub : Authentifiez-vous sur Docker Hub depuis votre terminal avec la commande :
docker login
2. Créer l'image en précisant un nom d'utilisateur pour l'unicité de l'image :

```
docker build -t <nom-utilisateur>/<nom-image>:<tag> .
```

3. Pusher l'image vers Docker Hub : Envoyez l'image taguée sur Docker Hub avec la commande docker push :

```
docker push <nom-utilisateur>/<nom-image>:<tag>
```

Vérifier sur Docker Hub : Une fois le push terminé, vous pouvez vérifier sur votre tableau de bord Docker Hub que l'image a bien été publiée. Elle sera désormais accessible publiquement (ou dans un dépôt privé si vous avez configuré un dépôt privé).

Sécurité sur Docker Hub

L'utilisation d'images publiques sur Docker Hub est très pratique, mais il est essentiel de prendre certaines précautions pour garantir la sécurité et la fiabilité de vos conteneurs.

- Créer vos propres images Si possible, créez vos propres images Docker à partir de Dockerfiles. Cela vous permet de contrôler chaque étape du processus de construction, de sélectionner les versions de logiciels que vous voulez utiliser et d'éviter les dépendances inutiles. Cela améliore la sécurité et la stabilité de vos conteneurs.
- Préférez les images officielles Les images officielles sont publiées par des éditeurs reconnus ou directement par Docker, garantissant un certain niveau de sécurité et de qualité. Par exemple, pour une base de données comme PostgreSQL, l'image officielle est régulièrement mise à jour et vérifiée pour éviter les vulnérabilités.

Sécurité sur Docker Hub

- Utiliser des tags spécifiques, pas “latest” Ne téléchargez pas systématiquement la version latest d’une image publique. Utilisez plutôt un tag spécifique pour garantir que vous utilisez une version testée et stable. Cela évite d’introduire des changements inattendus lors de la mise à jour des conteneurs.
- Scanner les images, même officielles Même si vous utilisez des images officielles, il est recommandé de les scanner avant utilisation. Des outils comme Docker Scan ou Clair permettent de détecter des failles potentielles ou des dépendances obsolètes dans les images.
- Contribuer à la communauté Si vous trouvez une vulnérabilité ou un problème dans une image publique, informez-en la communauté via les forums de Docker Hub ou proposez des corrections si l’image est open-source. Contribuer de cette manière améliore la sécurité de l’écosystème global.

Lors de la publication de vos images

Lorsque vous publiez des images sur Docker Hub, il est indispensable de prendre en compte certains aspects de sécurité, surtout pour les dépôts publics. Voici quelques bonnes pratiques pour protéger vos images Docker.

- Ne pas inclure de secrets dans vos images. N'intégrez jamais d'informations sensibles (comme des clés API, mots de passe ou fichiers de configuration) dans vos images Docker. Si une image est rendue publique, ces secrets seront accessibles à tous. Utilisez des variables d'environnement ou des gestionnaires de secrets comme Vault pour les gérer.
- Mettre à jour régulièrement vos images Maintenez vos images à jour pour corriger les failles de sécurité. En piochant régulièrement dans les dernières versions des dépendances et systèmes d'exploitation, vous garantisiez la sécurité de vos conteneurs en production.
- Scanner les images avant le push Utilisez des outils de scan de sécurité, comme Docker Scout ou des solutions externes comme Trivy, pour analyser vos images avant de les publier sur Docker Hub. Cela permet de détecter d'éventuelles vulnérabilités avant qu'elles ne soient exposées à vos utilisateurs.

Éviter les limitations de téléchargement depuis Docker Hub

Les comptes gratuits sur Docker Hub sont soumis à des restrictions de pulls d'images, avec une limite de 100 pulls toutes les 6 heures par adresse IP. Voici quelques stratégies pour contourner ces limitations sans passer à un compte payant.

- Optimiser le nombre de pulls Évitez de déclencher des pulls automatiques trop fréquents. Par exemple, dans vos pipelines CI/CD avec Gitlab CI, configurez un mécanisme pour vérifier si une image a déjà été téléchargée localement avant de la tirer depuis Docker Hub
- Utiliser un cache local Mettez en place un cache d'images Docker sur votre infrastructure. Un cache local évite de retélécharger les mêmes images depuis Docker Hub à chaque utilisation. Cela peut être réalisé en utilisant un registre privé avec des outils comme Harbor ou Nexus.

Ces techniques permettent de réduire l'impact des limites de pulls pour les comptes gratuits sur Docker Hub, tout en optimisant l'efficacité de vos workflows de conteneurs.

Alternatives à Docker Hub

Bien que Docker Hub soit la plateforme la plus populaire pour héberger et partager des images Docker, il existe plusieurs alternatives qui offrent des fonctionnalités similaires, voire supplémentaires, notamment pour ceux qui souhaitent éviter les limitations des comptes gratuits ou avoir un meilleur contrôle sur leurs images. Voici quelques alternatives à considérer.

GitLab CI

GitLab Container Registry offre un registry Docker intégré pour chaque projet. Cela permet de stocker et de gérer vos images directement au sein de votre infrastructure GitLab, avec une gestion complète des dépôts privés et un contrôle granulaire sur les accès. Il s'intègre parfaitement avec les pipelines CI/CD, ce qui facilite l'automatisation des workflows.

Amazon ECR

Amazon Elastic Container Registry (ECR) est un service géré par AWS pour stocker des images de conteneurs. Il est parfaitement adapté si vous utilisez d'autres services AWS comme EKS (Kubernetes). ECR permet de gérer des dépôts privés, offre un haut niveau de sécurité et est facilement intégrable avec des solutions CI/CD comme GitLab ou Jenkins.

GCR

Google Container Registry (GCR) est le service de registre de conteneurs de Google Cloud Platform (GCP). Il est conçu pour s'intégrer avec Google Kubernetes Engine (GKE) et d'autres services cloud de Google. GCR est robuste, sécurisé et peut être utilisé pour gérer des images privées à grande échelle, avec une tarification flexible selon l'usage.

Quay

Quay est un registre d'images Docker proposé par Red Hat. Il offre des fonctionnalités similaires à Docker Hub, mais avec une meilleure gestion des dépôts privés, des scans de sécurité intégrés et des options avancées pour l'automatisation. Quay propose également une version open-source que vous pouvez héberger localement.

ACR

Azure Container Registry (ACR) est le registre de conteneurs de Microsoft Azure. Comme ECR et GCR, il est conçu pour s'intégrer de manière transparente avec les services Azure tels que AKS (Azure Kubernetes Service). Il offre des fonctionnalités de sécurité avancées, des scans de vulnérabilités et la possibilité d'automatiser la gestion des images avec des pipelines CI/CD.

Harbor

Harbor est une solution open-source pour héberger des images Docker dans un registre privé. Il offre des fonctionnalités avancées comme la gestion des vulnérabilités, l'audit des images et le contrôle d'accès basé sur des rôles. C'est une option intéressante pour les entreprises souhaitant déployer leur propre registre et garder un contrôle total sur leurs images sans dépendre d'une plateforme externe.

Nexus

Sonatype Nexus Repository. est une solution populaire pour héberger non seulement des images Docker, mais aussi d'autres types d'artefacts comme les librairies Python et Maven. Il permet de gérer des dépôts privés et publics et propose des fonctionnalités avancées pour le contrôle d'accès, les scans de vulnérabilités, et l'automatisation des flux de travail. Nexus peut être auto-hébergé, offrant ainsi un contrôle total sur la gestion des images et artefacts et constitue une alternative flexible et extensible à Docker Hub.